

References

- Triplet Loss: [FaceNet: A Unified Embedding for Face Recognition and Clustering \(https://arxiv.org/pdf/1503.03832.pdf\)](https://arxiv.org/pdf/1503.03832.pdf)
- Sentence BERT: [entence-BERT: Sentence Embeddings using Siamese BERT-Networks \(https://arxiv.org/pdf/1908.10084.pdf\)](https://arxiv.org/pdf/1908.10084.pdf)

Embeddings

From our module on [Transfer Learning \(Transfer_Learning.ipynb\)](#) and [CLIP \(CLIP.ipynb\)](#)

- we understand that a Sequential Deep Learning model
- has multiple layers prior to the task-specific "head" (e.g., Classifier)
 - that create alternate representations of the Input
 - of increasing complexity as we go Deeper (closer to the Head)

The purpose of the layers from the Input to the Head

- is to create a representation
- from which the Head can correctly solve the task (e.g., predict the label)

These alternate representations are called *embeddings* of the Input.

We also hypothesize that

- embeddings that are close to the input
 - represent "syntax" concepts embeddings that are close to the Head
 - represent "semantic" concepts
 - but if we are too close to the Head
 - embeddings are over-specialized to the Source task used for training

We wish to understand behavior of embeddings visually.

Given a number of inputs

- we can compute the embedding of each
- and plot them in embedding space
- in order to understand the structure of embedding space

Here is a plot of the embeddings

- of a subset of the 10 digits

Although embeddings are typically high dimension

- we plot in 2D space
 - define by the first 2 Principal Components
- as a practical matter

And here is the clustering of text articles across different classes.

</table>

Attribution: <https://joeddav.github.io/blog/2020/05/29/ZSL.html>
(<https://joeddav.github.io/blog/2020/05/29/ZSL.html>)

It would seem

- that examples of the *same class* (e.g., digit or article topic)
- have embeddings that are close to one another
 - forming clusters in embedding space

This may explain why

- feeding embeddings into a Classifier head
- results in successful Classification

Using Embeddings to solve a task without finetuning

The traditional way that we have used Embeddings is via Transfer Learning

- keep a prefix of layers of a model trained for a Source task
 - use the embeddings created by the prefix
- append a new Head for a Target task
- train the Head on a small number of examples of the Target task
 - possibly also fine-tuning the weights in the prefix

Zero-shot classification

A simple approach to Classification via embeddings is like K-nearest neighbor

But, we could use the embeddings directly

- map an input $\mathbf{x}$ to its embedding $\text{embed}(\mathbf{x})$
- starting with a dataset of examples for a Target task
- predict $\mathbf{y}$, the label of $\mathbf{x}$
- create an equivalent dataset of embeddings
 - map each input to its embedding
 - as the label $\mathbf{y}^{ip}$ of an example $\langle \mathbf{x}^{ip}, \mathbf{y}^{ip} \rangle$

In this approach in a pre-defined set of labeled examples

$$[\langle \mathbf{x}^{ip}, \mathbf{y}^{ip} \rangle \mid 1 \leq i \leq m]$$

- whose embedding $\text{embed}(\mathbf{x}^{ip})$
- we don't modify (or need access to the source code) the prefix layers
- just treat the Source model as producing embeddings
 - is closest to $\text{embed}(\mathbf{x})$

$$\text{embed}(\mathbf{x}) \approx \text{embed}(\mathbf{x}^{ip})$$

This is motivated by the possibility

- that a new Target task
- can be solved *without training*
- just by comparing embeddings

Semantic search

Want to create your own search engine ?

- create text embeddings for each document in a collection
- create an embedding of your query

The document whose embedding is closest to the query's embedding would be the correct result.

Note

This is the basis for *Vector Stores*

- augmenting a LLM with your own data (e.g., GPT)

A simple example: facial (or image) recognition

- compare the embedding of an image
- with the embeddings of the fixed number of images for each class (e.g.,

Creating embeddings for similarity

Using the similarity of embeddings to solve a new Target task

- assumes that
- the embedding or "related" inputs
- are close to one another

We may get lucky and this will be the case for many tasks.

But we may need to train our embeddings

- with this as the specific goal
- by adding this as an objective of the Loss function

One such objective is the [Triplet Loss](https://arxiv.org/pdf/1503.03832.pdf) (<https://arxiv.org/pdf/1503.03832.pdf>)

Consider an input a (the "anchor")

Example: Sentence Embeddings

- with related input p ("positive")
- with unrelated input n ("negative")

To illustrate, we show [Sentence BERT](https://arxiv.org/pdf/1908.10084.pdf) (<https://arxiv.org/pdf/1908.10084.pdf>)

Let

- fine-tunes the embeddings produced by BERT
 - in order to make related sentences close in embedding space
 - $\|s - s'\|$ be a measure of the distance (inverse of similarity, always i
- between two embeddings s, s'

The Triplet Loss objective is to *minimize*

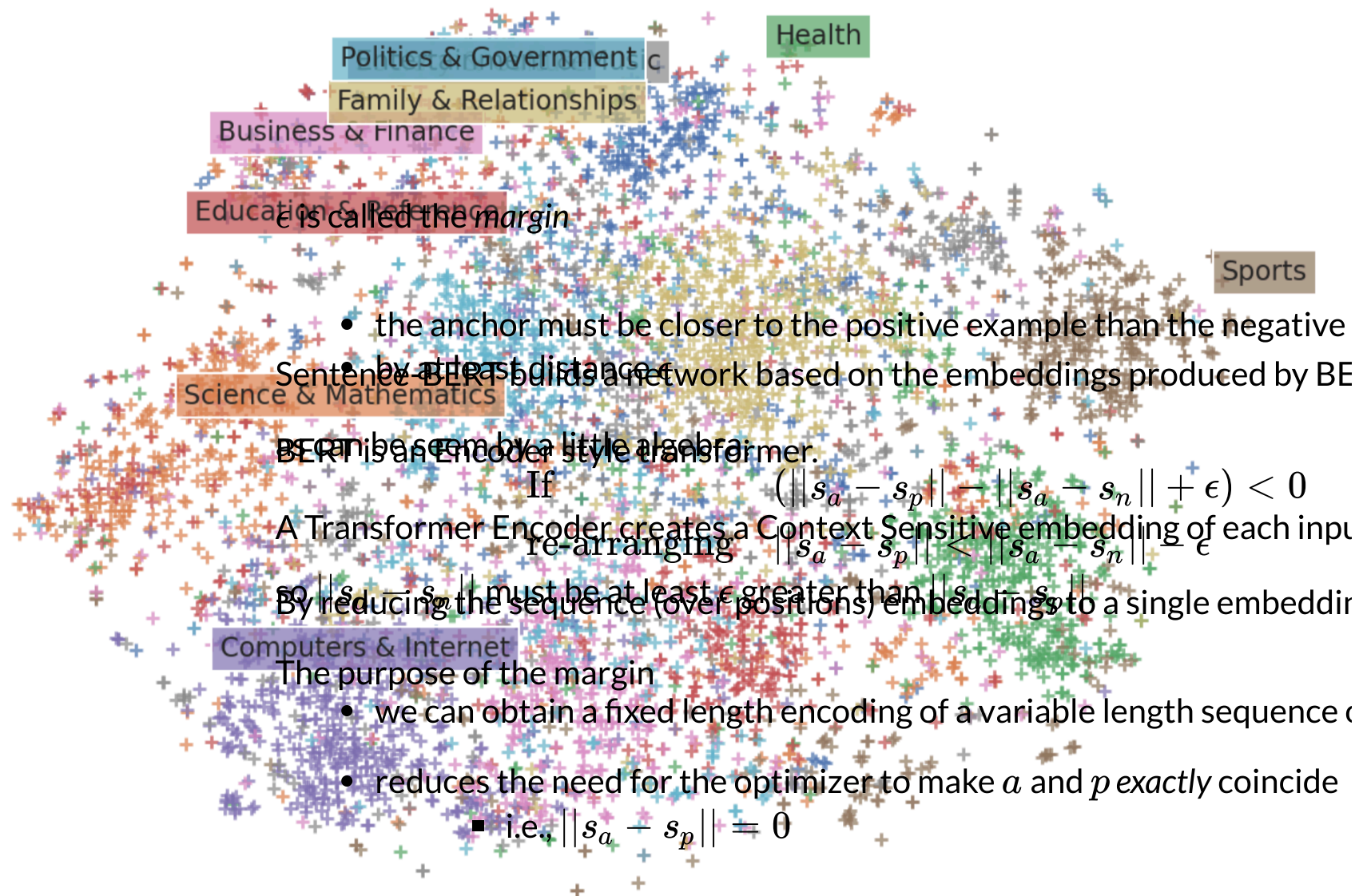
$$\max (||s_a - s_p|| - ||s_a - s_n|| + \epsilon, 0)$$

The loss is minimized when

- s_a is close to s_p
- s_a is far from s_n

That is the embedding for anchor

- a is very similar to that for p
- a is very dissimilar to that for n



</table>

Attribution: <https://arxiv.org/pdf/1908.10084.pdf#page=3>

(<https://arxiv.org/pdf/1908.10084.pdf#page=3>)

The pre-trained BERT model is *shared* across two inputs: Sentence A and Ser

- "weights are tied"
 - BERT's weights are fine-tuned via the Triplet Loss objective
- Aside

The sequence output of BERT is reduce by pooling (in this case),
Training a Source task with the Triplet Loss to perform the reduction of a sec
single value

- Sentence A is embedded as u
- produces embeddings
- Sentence B is embedded as v
- pooling (average over the embeddings)
- that are (usable for tasks that compare similarity of embeddings)

In the diagram on the right, the Triplet Objective
entire sequence

- using a beginning/end "special" token (e.g., $\langle \text{CLS} \rangle$) to capture the sur

- is recast as maximizing similarity (cosine distance)
- rather than minimizing distance

Aside

The diagram on the left is for producing embeddings for a specific task

- entailment
 - Does Sentence B logically follow from Sentence A
 - and hence is expressed as a Classification objective over labels $\{\text{"Entail"}, \text{"Does not entail"}\}$
- Here is the architecture

The inputs to the classifier are the concatenation of

- the embedding u of Sentence A
- the embedding v of Sentence B
- the difference in the embeddings

(Presumably these three inputs facilitate Classification)

The model is trained via batches that contain a mixture of

- Positive examples: Sentence A and Sentence B *are related* (anchor a at positive n)
- Negative examples: Sentence A and Sentence B *are un-related* (anchor a at positive n)

Triplet loss is minimized (or Utility maximized) in each batch.

Performance

[Here \(https://github.com/UKPLab/sentence-transformers/blob/master/docs/models/sts-models.md#performance-comparison\)](https://github.com/UKPLab/sentence-transformers/blob/master/docs/models/sts-models.md#performance-comparison) is a comparison of Sentence Transformers with other methods

The Sentence Embedding (Universal Sentence Encoder) scores highest

- outperforms Word Embeddings (the two GloVe entries)
- it *greatly outperforms* the simple reduction methods used on plain BERT
 - pooling (BERT as a service avg embeddings)
 - special <CLS> token (BERT as a service CLS vector)

Note

The "sophisticated" BERT, when using simple reduction methods

- underperforms the "old school" word embeddings !

In [6]:

Done